

RELATÓRIO EXPERIMENTO 2

Nome: Renato Gabriel Lima da Silva

Matrícula: 251027871

INTRODUÇÃO

Neste experimento, iremos abordar algumas entidades em VHDL e simular no software ModelSim, sendo elas um Somador completo com 3 bits de entrada (A,B e Cin) e 2 de saída (S e Cout) e um Multiplexador 4 para 1, vamos abordar seus conceitos e suas funcionalidades.

TEORIA

1 - Somador completo

No primeiro exercício vamos implementar um Somador completo, este somador é um circuito muito comum, onde ele soma as entradas (A,B e Cin) e apresenta o resultado da soma (S) e o chamado “vai um” (Cout) que é a forma como o circuito lida com o transbordamento aritmético quando a soma de três bits excede a capacidade máxima de representação de um único bit. Segue o circuito e a expressão que implementa um somador completo:

$$S = A \oplus B \oplus Cin$$
$$Cout = AB + ACin + BCin$$

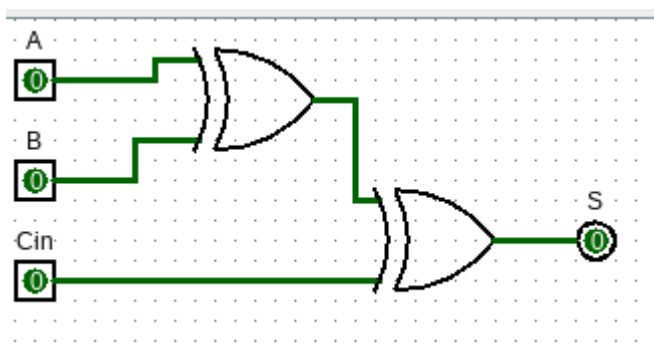


Figura 1: Circuito Soma do Somador completo no logisim

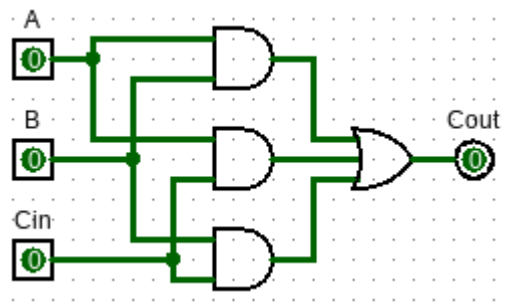


Figura 2 : Circuito Cout do Somador completo no logisim

2 - Multiplexador

No segundo exercício vamos implementar um Multiplexador 4x1, este circuito é um seletor de entradas, portanto o valor da saída (Y) vai depender da escolha feita pelas entradas de seleção (S_0 e S_1) das entradas de dados (D_0, D_1, D_2, D_3) Ex: caso S_0 e S_1 sejam 0 a saída será o valor apresentado em D_0 . Segue a expressão e circuito do Multiplexador:

$$Y = D_0 \bar{S}_1 \bar{S}_0 + D_1 \bar{S}_1 S_0 + D_2 S_1 \bar{S}_0 + D_3 S_1 S_0$$

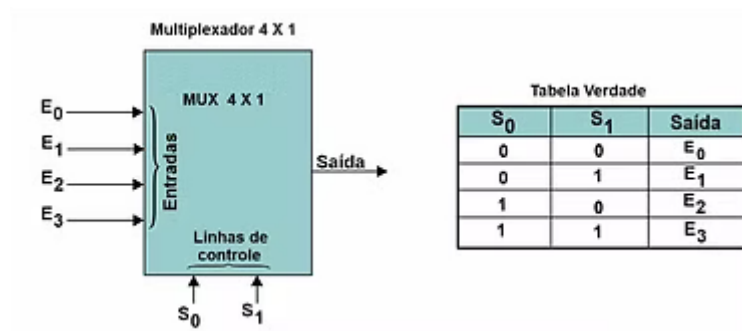


Figura 3 : exemplo de um Multiplexador e tabela verdade.

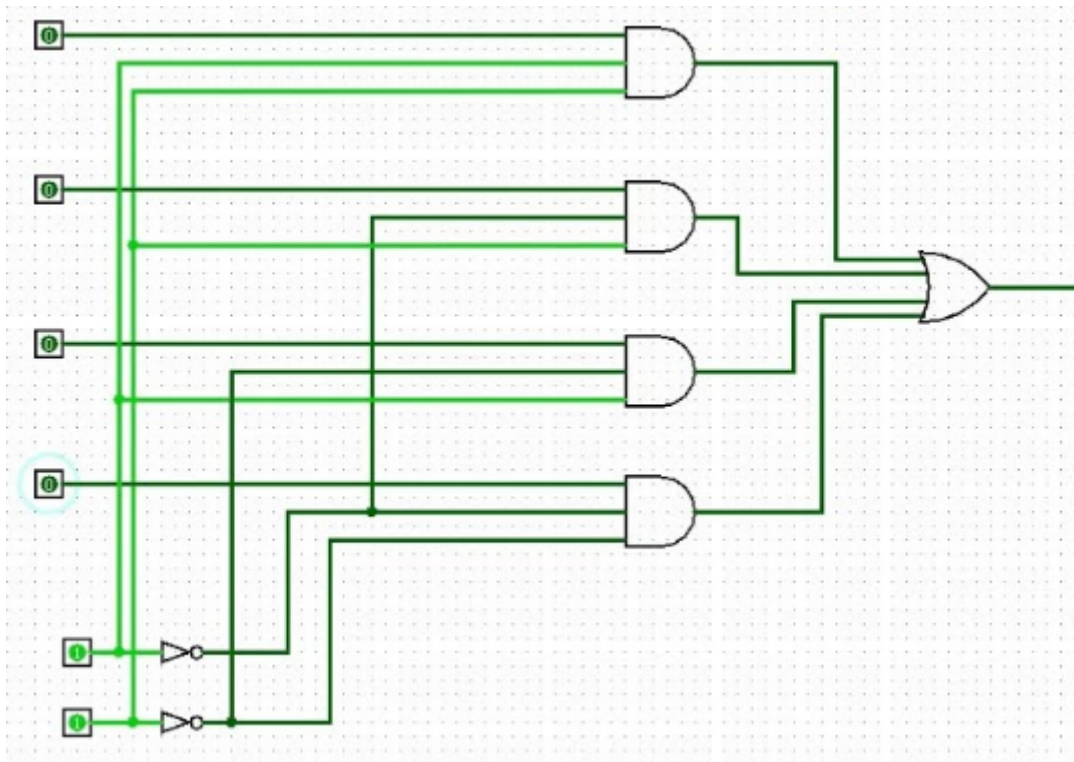


Figura 4: Circuito multiplexador no logisim

CÓDIGOS

1- Somador Completo

Neste experimento utilizamos a linguagem de VHDL por meio do software Modelsim para desenvolver o Somador completo e logo em seguida foi desenvolvido um código auxiliar chamado “testbench” para testarmos o circuito mudando os valores das entradas.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3
4  entity somador is
5      port(A,B,Cin: in std_logic;
6           S, Cout: out std_logic);
7  end somador;
8
9  architecture somador_arch of somador is
10 begin
11     S <= A xor B xor Cin;
12     Cout <= (A and B) or (A and Cin) or (B and Cin);
13
14 end somador_arch;

```

Figura 5: Código do Somador completo

Nas primeiras e segundas linhas definimos as bibliotecas que serão usadas no código no caso a “IEEE.STD_LOGIC_1164.all”. Em seguida, nas linhas 4 a 7, temos a definição de entidade do somador definindo as variáveis de entrada (A,B e Cin) e as de saídas (S e Cout). Depois nas linhas 9 a 14 temos por fim a definição da arquitetura do somador onde nas linhas 11 e 12 temos a implementação das funções apresentadas anteriormente.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3
4  entity tb_somador is
5  end tb_somador;
6
7  architecture teste of tb_somador is
8  component somador is
9      port(A,B,Cin: in std_logic;
10         S, Cout: out std_logic);
11  end component;
12
13  signal A,B,Cin: STD_LOGIC := '0';
14  signal S,Cout: STD_LOGIC;
15
16  begin
17
18  uut: somador port map(
19      A => A,
20      B => B,
21      Cin => Cin,
22      S => S,
23      Cout => Cout);
24
25  A <= not A after 40 ns;
26  B <= not B after 20 ns;
27  Cin <= not Cin after 10 ns;
28
29  end teste;
```

Figura 6: testbench do somador completo

Em seguida temos a implementação do testbench onde passamos a entidade definida anteriormente como um componente (linha 7 a 11) e

definimos valores para as entradas que variam em um determinado tempo ns (linha 25 a 27) para analisarmos as saídas.

2 - Multiplexador

Agora vamos analisar o código do multiplexador

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3
4  entity multiplexador is
5      port(S: in std_logic_vector (1 downto 0);
6           D: in std_logic_vector (3 downto 0);
7           Y: out std_logic);
8  end multiplexador;
9
10 architecture multiplexador_arch of multiplexador is
11 begin
12     Y<= (D(0)and(not S(1))and(not S(0))) or (D(1)and(not S(1))and S(0)) or
13         (D(2) and S(1) and(not S(0))) or (D(3) and S(1) and S(0));
14 end multiplexador_arch;
```

Figura 7 : Código do Multiplexador.

Neste código seguimos o mesmo esquema apresentado anteriormente. Primeiramente definimos as bibliotecas (linha 1 e 2). Em seguida, definimos a entidade do mux e definimos as entradas e saídas (linha 4 a 8), onde nesse caso para facilitar foi utilizado vetores onde na variável S possuímos o S(0) e o S(1) ou seja um vetor de tamanho 2, na variável D possuímos um vetor de tamanho 4 e por último a saída Y que não é um vetor. Depois na linha 9 a 13 definimos a arquitetura tomando muito cuidado com os parênteses e as portas not.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3
4  entity tb_mux is
5  end tb_mux;
6
7  architecture teste_mux of tb_mux is
8  component multiplexador is
9      port(S: in std_logic_vector (1 downto 0);
10         D: in std_logic_vector (3 downto 0);
11         Y: out std_logic);
12  end component;
13
14  signal S_tb : std_logic_vector(1 downto 0);
15  signal D_tb : std_logic_vector(3 downto 0);
16  signal Y_tb : std_logic;
17
18  begin
19
20  uut: multiplexador port map(
21      S => S_tb,
22      D => D_tb,
23      Y => Y_tb);
24
25  stimulus: process
26  begin
27
28      D_tb <= "0001";
29      S_tb <= "00";
30      wait for 10 ns;
31
32      S_tb <= "01";
33      wait for 10 ns;
34
35      S_tb <= "10";
36      wait for 10 ns;
37
38      S_tb <= "11";
39      wait for 10 ns;

```

```

41     D_tb <= "0010";
42     S_tb <= "00";
43     wait for 10 ns;
44
45     S_tb <= "01";
46     wait for 10 ns;
47
48     S_tb <= "10";
49     wait for 10 ns;
50
51     S_tb <= "11";
52     wait for 10 ns;
53
54     D_tb <= "0100";
55     S_tb <= "00";
56     wait for 10 ns;
57
58     S_tb <= "01";
59     wait for 10 ns;
60
61     S_tb <= "10";
62     wait for 10 ns;
63
64     S_tb <= "11";
65     wait for 10 ns;
66
67     D_tb <= "1000";
68     S_tb <= "00";
69     wait for 10 ns;
70
71     S_tb <= "01";
72     wait for 10 ns;
73
74     S_tb <= "10";
75     wait for 10 ns;
76
77     S_tb <= "11";
78     wait for 10 ns;
79     wait;
80
81 end process;
82 end teste_mux;

```

Figura 8: testbench do multiplexador.

No testbench do multiplexador foi utilizado apenas alguns casos como um multiplexador 4x1 possui muitos casos focamos apenas nos principais sendo portanto mudado apenas os valores de S e depois de todos os casos de S alteramos os valores de D, com os testes em D sendo 0001, 0010, 0100, 1000 assim conseguimos ver que os bits de seleção estão realmente funcionando corretamente e consequentemente o circuito está correto.

COMPILAÇÃO

Os códigos gerados anteriormente foram submetidos a uma compilação com o intuito de garantir o seu funcionamento:

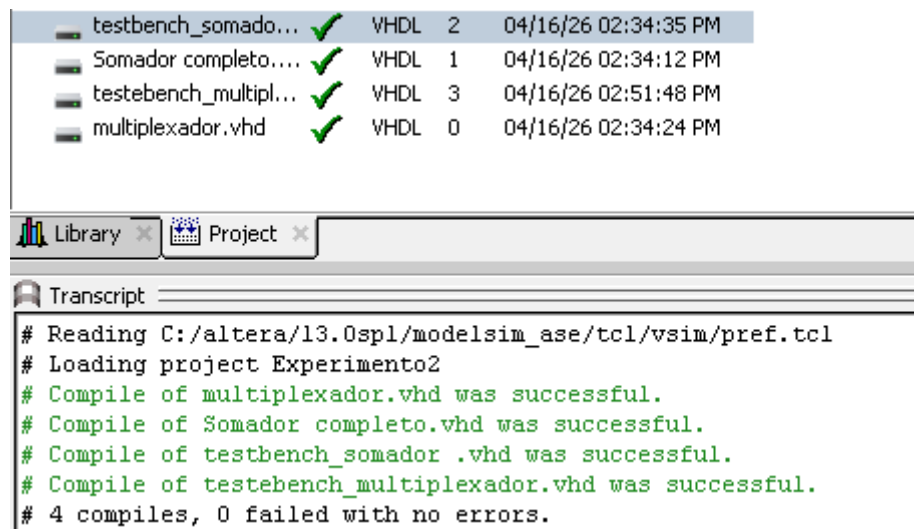


Figura 9: Compilação dos códigos

Como visto na figura 9 nenhum código tem algum erro de sintaxe.

SIMULAÇÃO E ANÁLISE

1- Somador Completo

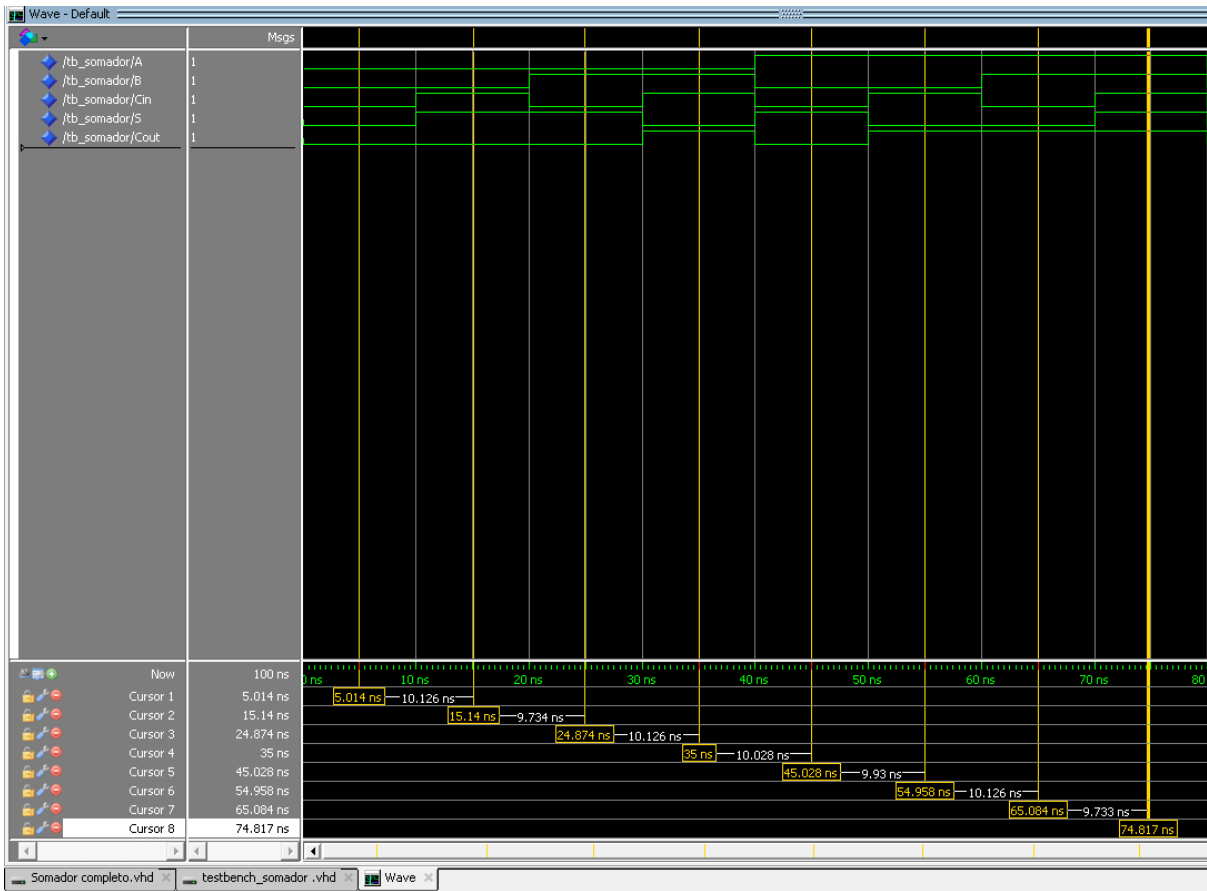


Figura 10: Simulação do Somador Completo em forma de onda binária

Entradas			Saídas	
A	B	C-in	C-out	S (Sum)
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Figura 11: Tabela verdade do Somador Completo

ANÁLISE

Como podemos ver o código conseguiu gerar corretamente a tabela verdade apresentada com cada cursor em um caso da tabela verdade ficando da seguinte maneira:

- Cursor 1: A=0, B=0, Cin = 0, S = 0 e Cout = 0;
- Cursor 2: A=0, B=0, Cin = 1, S = 1 e Cout = 0;
- Cursor 3: A=0, B=1, Cin = 0, S = 1 e Cout = 0;
- Cursor 4: A=0, B=1, Cin = 1, S = 0 e Cout = 1;
- Cursor 5: A=1, B=0, Cin = 0, S = 1 e Cout = 0;
- Cursor 6: A=1, B=0, Cin = 1, S = 0 e Cout = 1;
- Cursor 7: A=1, B=1, Cin = 0, S = 0 e Cout = 1;
- Cursor 8: A=1, B=1, Cin = 1, S = 1 e Cout = 1;

Neste caso podemos comprovar que o código se comporta como esperado gerando nosso somador completo.

2 - Multiplexador

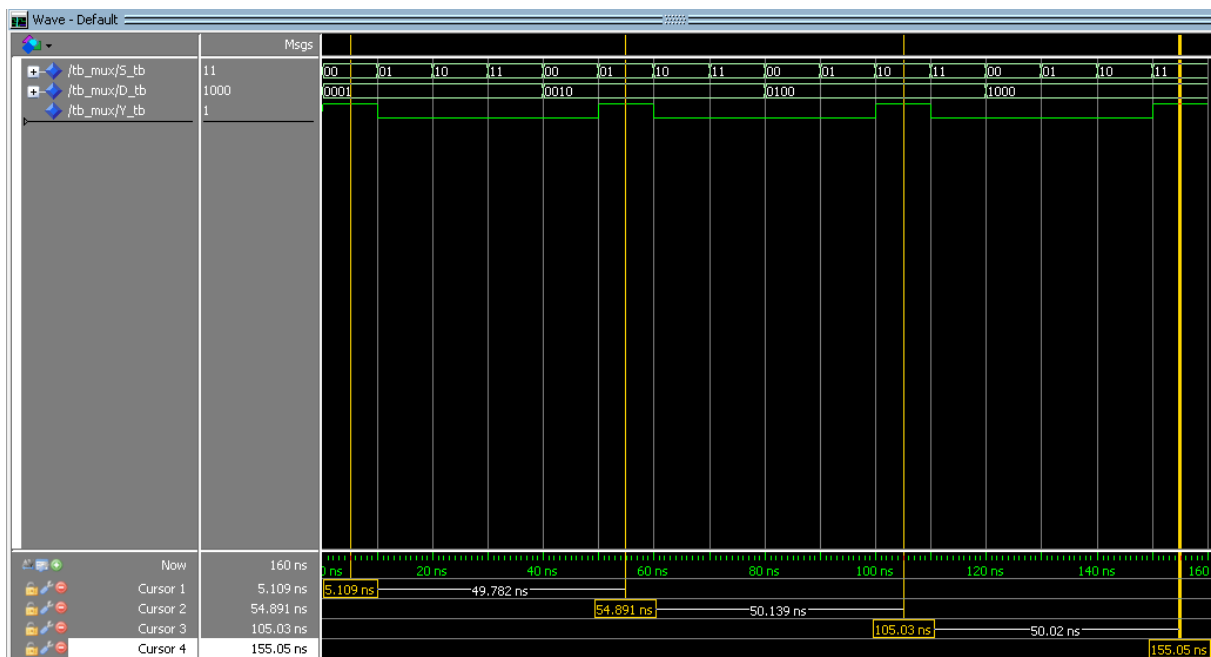


Figura 12: Simulação multiplexador em forma de onda binária

S_0	S_1	Saída
0	0	E_0
0	1	E_1
1	0	E_2
1	1	E_3

Figura 13: Tabela verdade Multiplexador sendo S_0 e S_1 os bits de seleção e E_0, E_1, E_2 e E_3 os dados.

ANÁLISE

Como podemos ver o código conseguiu gerar corretamente a tabela verdade apresentada com cada cursor em um caso onde o bit selecionado é um ficando da seguinte maneira:

- Cursor 1: $S = 00$, $D = 0001$, $Y = 1$
- Cursor 2: $S = 01$, $D = 0010$, $Y = 1$
- Cursor 3: $S = 10$, $D = 0100$, $Y = 1$
- Cursor 4: $S = 11$, $D = 1000$, $Y = 1$

Neste caso podemos comprovar que o código se comporta como esperado, pois com esses casos podemos comprovar que o código escolhe a saída correta a depender da seleção S .

CONCLUSÃO

Portanto, Neste experimento conseguimos conhecer a linguagem VHDL junto ao software ModelSim, onde conseguimos com êxito implementar duas funções e circuitos, sendo eles, o somador completo de 3 bits e um multiplexador de 4x1, ambos sendo compilados e simulados com o software, apresentado os mesmos resultados das tabelas verdades apresentadas. Não foram encontrados erros ou divergências neste experimento.